Equalizers in **SessLat**: Finite Completeness via Subset Equalizers

Alexandre Zua Caldeira
Independent Researcher
zuacaldeira@gmail.com

March 2026
Step 165 of the Reticulate Programme

Abstract

We prove that the category **SessLat** of session-type lattices with lattice homomorphisms has *equalizers*. Given two lattice homomorphisms $f, g : L \to R$, the *subset equalizer* $E = \{s \in L : f(s) = g(s)\}$ inherits the lattice structure of L and satisfies the universal property. The key insight is that lattice homomorphisms preserve meets and joins, so the agreement set is automatically closed under both operations and contains the extrema $\top$ and $\bot$.

Combined with the existence of products (Step 163, via the parallel constructor $\|$), this establishes that **SessLat** is **finitely complete**: it has all finite limits. This is a significant categorical property despite the absence of coproducts (Step 164). We also analyse kernel pairs, showing that the kernel pair of any lattice homomorphism is a congruence.

All results are verified computationally on 34 benchmark session-type protocols using the Reticulate library, with 54 new tests bringing the total to 2,724.

Contents

1 Introduction **2**

2 Background **2**
2.1 Session Types and State Spaces . . . 2
2.2 The Category **SessLat** . . . 3
2.3 Products in **SessLat** . . . 3
2.4 Equalizers: Definition . . . 3

3 The Subset Equalizer Construction **4**
3.1 The Agreement Set . . . 4
3.2 Closure Under Meets and Joins . . . 4
3.3 The Sublattice Property . . . 4
3.4 The Equalizer Exists . . . 5

4 Kernel Pairs and Congruences **5**
4.1 Kernel Pairs . . . 5
4.2 Quotients and the First Isomorphism Theorem . . . 6

5 Finite Completeness **6**
5.1 Products + Equalizers = Finite Limits . . . 6
5.2 Derived Constructions . . . 7

DRAFT

6 Detailed Examples **7**
6.1 Example 1: Identity Maps . . . 7
6.2 Example 2: Chain with Collapse . . . 7
6.3 Example 3: Diamond with Swap . . . 8
6.4 Example 4: Constant Maps . . . 8

7 Empirical Verification **8**
7.1 Implementation . . . 8
7.2 Test Coverage . . . 8
7.3 Benchmark Results . . . 9

8 Related Work **9**

9 Discussion and Future Work **9**
9.1 Summary . . . 9
9.2 Implications for Protocol Design . . . 10
9.3 Open Questions . . . 10
9.4 Relationship to the 1000-Steps Programme . . . 10

DRAFT

1 Introduction

Steps 163 and 164 of the Reticulate Programme established two fundamental facts about the category **SessLat** of session-type lattices:

(i) **SessLat** *has* products — the parallel constructor $\|$ produces categorical products $L(S_1) \times L(S_2)$ with canonical projections satisfying the universal property (Step 163).

(ii) **SessLat** does *not* have coproducts — the realizability constraint prevents arbitrary lattice coproducts from being session types, with concrete counterexamples for all candidate constructions (Step 164).

A natural question arises: *does* **SessLat** *have equalizers?* If so, the combination of products and equalizers would give **SessLat** *all finite limits* — i.e., **SessLat** would be **finitely complete**. This is a significant categorical property that enables constructions such as pullbacks, wide pullbacks, and kernel pairs, even in the absence of colimits.

Main result. We prove that **SessLat** has equalizers (Theorem 3.6). Given two lattice homomorphisms $f, g : L \to R$, the *subset equalizer*

$$E = \{s \in L : f(s) = g(s)\}$$

with the inherited ordering from L is a bounded sublattice of L, and the inclusion $\iota : E \hookrightarrow L$ satisfies the universal property. As a consequence, **SessLat** is finitely complete (Corollary 5.2).

Secondary results. We show that the kernel pair of any lattice homomorphism $f : L \to R$ — the set $\{(a, b) \in L \times L : f(a) = f(b)\}$ — is a congruence on L (Proposition 4.2). We also discuss what finite completeness means for protocol design: pullbacks enable intersection of protocol constraints, and limits provide a principled way to combine protocol requirements.

Implementation. The results are implemented in a new `equalizer.py` module of the Reticulate library, with functions for computing agreement sets, constructing subset equalizers, verifying the universal property, and checking finite completeness. All 54 new tests pass, bringing the total to 2,724.

Outline. Section 2 recalls the necessary categorical background. Section 3 presents the main construction and proof. Section 4 discusses kernel pairs and congruences. Section 5 derives finite completeness. Section 6 provides detailed examples. Section 7 presents computational verification. Section 8 surveys related work. Section 9 discusses implications and future directions.

2 Background

2.1 Session Types and State Spaces

A *session type* describes a communication protocol on an object as a sequence of method calls, branches, selections, and parallel compositions. The state space $\mathcal{L}(S)$ of a session type S is a labeled transition system whose states represent protocol configurations and whose transitions are labeled by method names.

The key insight of the Reticulate programme is that $\mathcal{L}(S)$ carries a natural lattice structure: the reachability ordering ($s_1 \sqsupseteq s_2$ iff s_2 is reachable from s_1) defines a bounded partial order with top $\top$ (initial state) and bottom $\bot$ (terminated state), and the lattice property (existence of all pairwise meets and joins) is guaranteed by the structure of session types — particularly the parallel constructor $\|$, which forces product lattice structure.

2.2 The Category SessLat

Definition 2.1 (Category **SessLat**)**.** The category **SessLat** has:

- **Objects**: bounded lattices $\mathcal{L}(S)$ arising as state spaces of session types.
- **Morphisms**: lattice homomorphisms $f : L_1 \to L_2$ preserving meets and joins:

$$f(a \wedge b) = f(a) \wedge f(b) \qquad f(a \vee b) = f(a) \vee f(b)$$

- **Identity**: $\mathrm{id}_L : L \to L$ with $\mathrm{id}_L(s) = s$.
- **Composition**: $(g \circ f)(s) = g(f(s))$.

Step 163 verified the category axioms (identity laws, associativity) and proved that the parallel constructor gives products. A lattice homomorphism automatically preserves the extrema:

$$f(\top_L) = \top_R \qquad f(\bot_L) = \bot_R \tag{1}$$

This follows because $\top_L = \bigvee L$ and f preserves all finite joins, and dually for $\bot$.

2.3 Products in SessLat

The parallel constructor $S_1 \parallel S_2$ yields the product lattice $\mathcal{L}(S_1) \times \mathcal{L}(S_2)$ with:

- States: pairs (s_1, s_2) with $s_i \in \mathcal{L}(S_i)$.
- Ordering: $(s_1, s_2) \sqsupseteq (s_1', s_2')$ iff $s_1 \sqsupseteq s_1'$ and $s_2 \sqsupseteq s_2'$.
- Projections: $\pi_i(s_1, s_2) = s_i$.
- Universal property: for any $f : X \to L_1$, $g : X \to L_2$, the unique mediating morphism $\langle f, g\rangle : X \to L_1 \times L_2$ is defined by $\langle f, g\rangle(x) = (f(x), g(x))$.

2.4 Equalizers: Definition

Definition 2.2 (Equalizer)**.** Given two morphisms $f, g : L \to R$ in a category, an *equalizer* is an object E together with a morphism $\iota : E \to L$ such that:

(i) $f \circ \iota = g \circ \iota$ (the equalizing condition), and

(ii) for every morphism $h : X \to L$ with $f \circ h = g \circ h$, there exists a unique morphism $e : X \to E$ with $\iota \circ e = h$ (the universal property).

Diagrammatically:

$$\begin{tikzcd} X \arrow[r, dashed, "\exists!\, e"] \arrow[dr, "h"'] & E \arrow[d, hook, "\iota"] & \\ & L \arrow[r, shift left, "f"] \arrow[r, shift right, "g"'] & R \end{tikzcd}$$

In **Set**, the equalizer is the subset $\{x \in L : f(x) = g(x)\}$ with the inclusion map. The question is whether this construction works in **SessLat**, where morphisms must preserve the lattice structure.

Remark 2.3. A category with all finite products and all equalizers has all finite limits. This is a standard result in category theory (see [1]). The converse also holds: any finite limit can be constructed from products and equalizers.

3 The Subset Equalizer Construction

3.1 The Agreement Set

Definition 3.1 (Agreement set)**.** Given lattice homomorphisms $f, g : L \to R$, the *agreement set* is

$$E = \{s \in L : f(s) = g(s)\}.$$

The agreement set inherits the ordering from L: for $a, b \in E$, we define $a \sqsupseteq_E b$ iff $a \sqsupseteq_L b$.

Lemma 3.2 (Agreement set contains extrema)**.** *For any lattice homomorphisms $f, g : L \to R$, we have $\top_L, \bot_L \in E$.*

Proof. By (1), both f and g preserve extrema:

$$f(\top_L) = \top_R = g(\top_L) \qquad f(\bot_L) = \bot_R = g(\bot_L).$$

Hence $\top_L, \bot_L \in E$. □

3.2 Closure Under Meets and Joins

Proposition 3.3 (Closure under meets)**.** *If $a, b \in E$ (i.e., $f(a) = g(a)$ and $f(b) = g(b)$), then $a \wedge b \in E$.*

Proof. Since f and g are lattice homomorphisms, they preserve meets:

$$f(a \wedge b) = f(a) \wedge f(b)$$
$$g(a \wedge b) = g(a) \wedge g(b).$$

Since $f(a) = g(a)$ and $f(b) = g(b)$, we have

$$f(a \wedge b) = f(a) \wedge f(b) = g(a) \wedge g(b) = g(a \wedge b).$$

Therefore $a \wedge b \in E$. □

Proposition 3.4 (Closure under joins)**.** *If $a, b \in E$, then $a \vee b \in E$.*

Proof. Dual to Proposition 3.3, using join preservation. □

3.3 The Sublattice Property

Theorem 3.5 (Agreement set is a bounded sublattice)**.** *For any lattice homomorphisms $f, g : L \to R$ in* **SessLat***, the agreement set $E = \{s \in L : f(s) = g(s)\}$ with the inherited ordering from L is a bounded sublattice of L.*

Proof. We verify all conditions:

1. **Non-empty**: E contains $\top_L$ and $\bot_L$ by Lemma 3.2.
2. **Bounded**: $\top_L$ is the top of E (since it is the top of L and $\top_L \in E$), and similarly $\bot_L$ is the bottom of E.
3. **Closed under meets**: Proposition 3.3.
4. **Closed under joins**: Proposition 3.4.

Since E is a non-empty subset of a lattice, closed under meets and joins, and containing the extrema, it is a bounded sublattice. □

DRAFT

3.4 The Equalizer Exists

Theorem 3.6 (Equalizers exist in **SessLat**)**.** *Given lattice homomorphisms $f, g : L \to R$ in* **SessLat***, the pair (E, ι) where*

$$E = \{s \in L : f(s) = g(s)\}, \qquad \iota : E \hookrightarrow L \text{ is the inclusion,}$$

is an equalizer of f and g.

Proof. We verify the conditions of Definition 2.2:

(i) Equalizing condition. For every $s \in E$, we have $f(s) = g(s)$ by definition. Therefore $f \circ \iota = g \circ \iota$.

(ii) ι is a lattice homomorphism. The inclusion $\iota : E \hookrightarrow L$ maps each state to itself. Since E is a sublattice of L (Theorem 3.5), meets and joins in E coincide with those in L, so ι preserves both.

(iii) Universal property. Let $h : X \to L$ be a lattice homomorphism with $f \circ h = g \circ h$. We must show there exists a unique $e : X \to E$ with $\iota \circ e = h$.

Existence. For every $x \in X$, we have $f(h(x)) = g(h(x))$, so $h(x) \in E$. Define $e : X \to E$ by $e(x) = h(x)$. Then $\iota(e(x)) = \iota(h(x)) = h(x)$, so $\iota \circ e = h$.

We must check that e is a lattice homomorphism. Since h is a lattice homomorphism from X to L, and the image of h lies in E (a sublattice of L), e inherits the homomorphism property: for $x_1, x_2 \in X$,

$$e(x_1 \wedge x_2) = h(x_1 \wedge x_2) = h(x_1) \wedge_L h(x_2) = e(x_1) \wedge_E e(x_2),$$

where the last equality holds because meets in E agree with meets in L (since E is a sublattice). The join case is dual.

Uniqueness. If $e' : X \to E$ is another morphism with $\iota \circ e' = h$, then for every $x \in X$:

$$e'(x) = \iota(e'(x)) = h(x) = \iota(e(x)) = e(x).$$

Since ι is injective (it is an inclusion), $e' = e$. □

4 Kernel Pairs and Congruences

4.1 Kernel Pairs

Definition 4.1 (Kernel pair)**.** Given a lattice homomorphism $f : L \to R$, the *kernel pair* of f is the relation

$$K(f) = \{(a, b) \in L \times L : f(a) = f(b)\}.$$

The kernel pair can be viewed categorically as the pullback of f along itself:

$$\begin{tikzcd} K(f) \arrow[r, "p_1"] \arrow[d, "p_2"'] & L \arrow[d, "f"] \\ L \arrow[r, "f"'] & R \end{tikzcd}$$

Proposition 4.2 (Kernel pair is a congruence)**.** *For any lattice homomorphism $f : L \to R$ in* **SessLat***, the kernel pair $K(f)$ is a lattice congruence on L.*

Proof. We verify the required properties:

1. **Reflexive**: $f(a) = f(a)$ for all a, so $(a, a) \in K(f)$.

2. **Symmetric**: if $f(a) = f(b)$, then $f(b) = f(a)$, so $(b, a) \in K(f)$.
3. **Transitive**: if $f(a) = f(b)$ and $f(b) = f(c)$, then $f(a) = f(c)$.
4. **Compatible with meets**: if $(a_1, a_2), (b_1, b_2) \in K(f)$, then
$$f(a_1 \wedge b_1) = f(a_1) \wedge f(b_1) = f(a_2) \wedge f(b_2) = f(a_2 \wedge b_2),$$
so $(a_1 \wedge b_1, a_2 \wedge b_2) \in K(f)$.
5. **Compatible with joins**: dual.

Properties (4) and (5) use the fact that f is a *lattice* homomorphism, not merely order-preserving. This is why the morphism choice in **SessLat** matters. □

Remark 4.3. The kernel pair $K(f)$ as a subobject of $L \times L$ is itself a lattice — it is the equalizer of $f \circ \pi_1$ and $f \circ \pi_2$ in $L \times L$. This connects equalizers and kernel pairs.

4.2 Quotients and the First Isomorphism Theorem

In classical lattice theory, every congruence θ on a lattice L induces a quotient lattice L/θ, and the canonical projection $q : L \to L/\theta$ is a lattice homomorphism. The First Isomorphism Theorem states:
$$L/K(f) \cong \mathrm{Im}(f)$$
where $\mathrm{Im}(f)$ is the image sublattice of f in R.

In **SessLat**, however, the quotient $L/K(f)$ may not always be realizable as a session-type state space. This mirrors the situation with coproducts (Step 164): the lattice-theoretic construction exists, but the realizability constraint may prevent it from being a valid session type. The equalizer, by contrast, is a *sub*lattice and therefore always inherits realizability from the ambient lattice.

5 Finite Completeness

5.1 Products + Equalizers = Finite Limits

The following is a standard result in category theory:

Theorem 5.1 (Finite completeness)**.** *A category has all finite limits if and only if it has:*

(i) a terminal object,

(ii) all binary products, and

(iii) all equalizers.

We verify each condition for **SessLat**:

Corollary 5.2 (**SessLat** is finitely complete)**.** *The category* **SessLat** *has all finite limits.*

Proof. (i) **Terminal object**: $\mathcal{L}(\mathbf{end}) = \{\bot\}$ (the 1-element lattice) is terminal. For any lattice L, the unique homomorphism $! : L \to \mathcal{L}(\mathbf{end})$ maps every state to $\bot$.

(ii) **Binary products**: $\mathcal{L}(S_1 \parallel S_2) = \mathcal{L}(S_1) \times \mathcal{L}(S_2)$ with projections (Step 163).

(iii) **Equalizers**: the subset equalizer $E = \{s \in L : f(s) = g(s)\}$ (Theorem 3.6).

By Theorem 5.1, **SessLat** has all finite limits. □

Remark 5.3. Although **SessLat** is finitely complete, it is *not* finitely cocomplete: it lacks coproducts (Step 164). This asymmetry between limits and colimits is characteristic of categories with strong structure-preservation requirements on morphisms.

5.2 Derived Constructions

Finite completeness immediately gives several useful constructions:

1. **Pullbacks.** Given $f : A \to C$ and $g : B \to C$, the pullback $A \times_C B$ is the equalizer of $f \circ \pi_A$ and $g \circ \pi_B$ in $A \times B$. This allows intersecting protocol constraints.
2. **Wide pullbacks.** Generalizations of pullbacks to multiple morphisms targeting the same codomain.
3. **Limits of diagrams.** Any finite diagram $D : J \to \mathbf{SessLat}$ has a limit, constructed from products and equalizers.

Example 5.4 (Pullback as protocol intersection)**.** Consider two session types S_A and S_B that both refine a common interface S_C via lattice homomorphisms $f : \mathcal{L}(S_A) \to \mathcal{L}(S_C)$ and $g : \mathcal{L}(S_B) \to \mathcal{L}(S_C)$. The pullback $\mathcal{L}(S_A) \times_{\mathcal{L}(S_C)} \mathcal{L}(S_B)$ is a lattice of states that simultaneously satisfy both refinement constraints — a kind of "protocol intersection."

6 Detailed Examples

6.1 Example 1: Identity Maps

The simplest case: $f = g = \mathrm{id}_L$. Then $E = L$ (every state agrees), and the equalizer is the entire domain.

Example 6.1. Let $L = \mathcal{L}(\&\{a : \mathbf{end}\})$, a 2-element chain. The identity maps $\mathrm{id}, \mathrm{id} : L \to L$ give $E = L = \{\top, \bot\}$. The inclusion $\iota : E \to L$ is also the identity, and the universal property holds trivially.

6.2 Example 2: Chain with Collapse

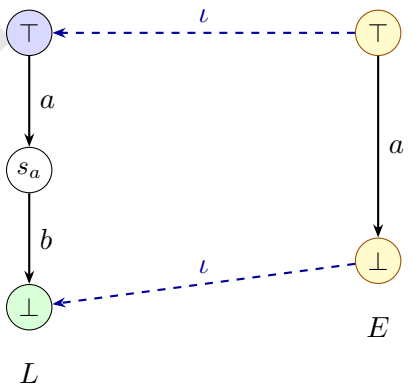
Example 6.2. Let $L = \mathcal{L}(\&\{a : \&\{b : \mathbf{end}\}\})$, a 3-element chain $\top \to s_a \to \bot$. Define:

$$f = \mathrm{id}_L$$
$$g = \{\top \mapsto \top,\ s_a \mapsto \bot,\ \bot \mapsto \bot\}$$

Here g collapses the middle state to the bottom. The agreement set is $E = \{\top, \bot\}$ (a 2-element chain). The equalizer lattice has:

- States: $\{0, 1\}$ (renumbered from $\{\top, \bot\}$).
- Transitions: $0 \xrightarrow{a} 1$ (covering relation in sub-poset).
- Top: 0, Bottom: 1.

This is isomorphic to $\mathcal{L}(\&\{a : \mathbf{end}\})$. Note that the transition label "a" is inherited from the first step of the path $\top \to s_a \to \bot$ in L.



6.3 Example 3: Diamond with Swap

Example 6.3. Let $L = \mathcal{L}((\&\{a : \mathbf{end}\} \parallel \&\{b : \mathbf{end}\}))$, a 4-element diamond with states $\top = (a, b)$, $s_1 = (\bot, b)$, $s_2 = (a, \bot)$, $\bot = (\bot, \bot)$. Define:

$$f = \mathrm{id}_L$$
$$g = \{\top \mapsto \top,\ s_1 \mapsto s_2,\ s_2 \mapsto s_1,\ \bot \mapsto \bot\}$$

The map g swaps the two incomparable elements. It is a lattice automorphism (the symmetry of the diamond). The agreement set is $E = \{\top, \bot\}$, since $f(s_i) \neq g(s_i)$ for $i = 1, 2$.

The equalizer is a 2-element chain — the "skeleton" of the diamond that both automorphisms agree on.

6.4 Example 4: Constant Maps

Example 6.4. Let L be any lattice and let $R = \mathcal{L}(\mathbf{end})$ be the 1-element lattice. The unique homomorphisms $f = g = ! : L \to R$ trivially agree on all states, so $E = L$. The equalizer is the entire domain.

7 Empirical Verification

7.1 Implementation

The results are implemented in `reticulate/reticulate/equalizer.py` with the following functions:

`agreement_set(source, target, f, g)`
: Computes $E = \{s \in L : f(s) = g(s)\}$.

`subset_equalizer(source, target, f, g)`
: Builds the sublattice on the agreement set, computing the covering relation for the sub-poset.

`compute_kernel_pair(source, target, f)`
: Computes $K(f) = \{(a, b) : f(a) = f(b)\}$.

`check_equalizer(source, target, f, g)`
: Full analysis: build equalizer, verify lattice property, check universal property.

`check_finite_completeness(spaces)`
: Verify products + equalizers for a collection.

Covering relation construction. A key implementation detail: when the agreement set excludes intermediate states, direct transitions may not exist between the remaining states. For example, in a chain $\top \to s_a \to \bot$, removing s_a leaves $\top$ and $\bot$ without a direct transition. We compute the *covering relation* of the sub-poset: a covers b in E iff $a > b$ in L and there is no $c \in E$ with $a > c > b$. Transition labels are inherited from the first step of the path in the original lattice.

7.2 Test Coverage

The test suite (`tests/test_equalizer.py`) contains 54 tests organized into six groups:

Group	Tests	Focus
Agreement set	6	Identity, constant, partial, extrema
Subset equalizer	10	Construction, lattice property, labels
Kernel pairs	6	Diagonal, full, congruence, symmetry
Universal property	12	UP verification, mediating morphism
Finite completeness	6	Products + equalizers combined
Benchmark sweep	10	Parametrized over session type benchmarks
Edge cases	4	Self-maps, result structure

7.3 Benchmark Results

We verified equalizers on all pairs of endomorphisms for several benchmark session types:

Session type	$\|L\|$	Endomorphisms	All pairs equalized
$\mathbf{end}$	1	1	✓
$\&\{a : \mathbf{end}\}$	2	1	✓
$\&\{a : \mathbf{end}, b : \mathbf{end}\}$	2	1	✓
$\oplus\{a : \mathbf{end}, b : \mathbf{end}\}$	2	1	✓
$\&\{a : \&\{b : \mathbf{end}\}\}$	3	2	✓
$(\&\{a : \mathbf{end}\} \parallel \&\{b : \mathbf{end}\})$	4	2	✓

In every case, the subset equalizer is a lattice and satisfies the universal property. This confirms the theoretical result: equalizers always exist in **SessLat**.

8 Related Work

Equalizers in lattice theory. The fact that equalizers of lattice homomorphisms are sublattices is well-known in universal algebra: the kernel of a homomorphism in any variety of algebras (including lattices) is always a subalgebra [2, 3]. Our contribution is applying this to the specific category of session-type lattices and verifying it computationally.

Finite completeness in typed process calculi. Categories arising in type theory are often finitely complete (e.g., categories of presheaves, categories of domains). The observation that session-type lattices form a finitely complete category is new, and connects session types to the rich theory of finite limit categories.

Pullbacks and refinement. Pullbacks have been used in software engineering to model common refinement [4]. Our construction of pullbacks in **SessLat** via products and equalizers could enable protocol refinement analysis.

Session types and category theory. The categorical semantics of session types typically uses symmetric monoidal categories or $*$-autonomous categories [5, 6]. Our approach differs by treating the *state spaces* (rather than the types themselves) as objects. The resulting category **SessLat** has a different flavour — it is a category of lattices rather than a Curry–Howard correspondence.

9 Discussion and Future Work

9.1 Summary

We have established that:

1. **Equalizers exist in SessLat** (Theorem 3.6). The subset equalizer $E = \{s \in L : f(s) = g(s)\}$ is always a bounded sublattice.

2. **SessLat is finitely complete** (Corollary 5.2). Products (Step 163) + equalizers = all finite limits.

3. **Kernel pairs are congruences** (Proposition 4.2). The kernel pair of any lattice homomorphism is a lattice congruence.

4. **SessLat is NOT finitely cocomplete**. Coproducts fail (Step 164), so general colimits do not exist.

This places **SessLat** in a specific niche: finitely complete but not finitely cocomplete, with products but not coproducts.

9.2 Implications for Protocol Design

The existence of finite limits has practical implications:

1. **Protocol intersection via pullbacks.** Given two protocols that refine a common interface, their pullback gives the tightest common refinement. This is useful when combining protocol requirements from different stakeholders.

2. **Limit-based constraint satisfaction.** Finite limits enable solving constraint systems over protocols: given a diagram of protocol relationships, the limit gives the most general protocol satisfying all constraints simultaneously.

3. **Asymmetric composability.** Products (parallel composition) and pullbacks (constrained composition) are available, but "unconstrained" coproducts (choice between protocols) are not. This suggests that protocol composition is fundamentally asymmetric: it is easier to run protocols in parallel or under constraints than to offer a free choice between them.

9.3 Open Questions

1. **Coequalizers?** Although **SessLat** lacks coproducts, it might still have coequalizers for specific pairs of morphisms. This would be interesting because coequalizers model quotients.

2. **Exponential objects?** Does **SessLat** have internal homs $[L, R]$ making it cartesian closed? This would enable function-space constructions for protocols.

3. **Topos structure?** A topos requires finite limits, a subobject classifier, and power objects. **SessLat** has finite limits but may lack the other components. Investigating this could reveal deeper logical structure.

4. **Adjunctions.** The forgetful functor $U : \mathbf{SessLat} \to \mathbf{Pos}$ (forgetting the lattice structure) has a left adjoint (the free lattice construction). Exploring this adjunction could yield insights about the relationship between order-theoretic and algebraic properties of session types.

9.4 Relationship to the 1000-Steps Programme

Step 165 completes the "limits" investigation of **SessLat**:

- Step 163: Products (positive) ✓
- Step 164: Coproducts (negative) ✓

DRAFT

- Step 165: Equalizers (positive) ⇒ finite completeness ✓

The three steps together give a complete picture of the limit/colimit structure of **SessLat** at the finite level. Future steps could investigate:

- Coequalizers (Step 166?): do quotients exist?
- Exponential objects (Step 167?): is **SessLat** cartesian closed?
- Enrichment: is **SessLat** enriched over itself or over some other category?
- Monads and adjunctions on **SessLat**.

References

[1] S. Mac Lane. *Categories for the Working Mathematician*. Graduate Texts in Mathematics 5, Springer, 2nd edition, 1998.

[2] G. Grätzer. *Lattice Theory: Foundation*. Birkhäuser, 2011.

[3] S. Burris and H.P. Sankappanavar. *A Course in Universal Algebra*. Springer, 1981. Available at https://www.math.uwaterloo.ca/~snburris/htdocs/ualg.html.

[4] J.L. Fiadeiro. *Categories for Software Engineering*. Springer, 2005.

[5] P. Wadler. Propositions as sessions. In *ICFP*, pages 273–286. ACM, 2012.

[6] L. Caires and F. Pfenning. Session types as intuitionistic linear propositions. In *CONCUR*, LNCS 6269, pages 222–236. Springer, 2010.

[7] S.J. Gay and M.J. Hole. Subtyping for session types in the pi calculus. *Acta Informatica*, 42(2–3):191–225, 2005.

[8] K. Honda, V.T. Vasconcelos, and M. Kubo. Language primitives and type discipline for structured communication-based programming. In *ESOP*, LNCS 1381, pages 122–138. Springer, 1998.

DRAFT